

Akademia Górniczo-Hutnicza
im. Stanisława Staszica w Krakowie

Wydział Informatyki

Algorytmy Geometryczne

Lokalizacja punktu w przestrzeni dwuwymiarowej –
metoda separatorów

Dokumentacja

1 Wprowadzenie

Niniejsze opracowanie stanowi dokumentację projektu realizowanego w ramach przedmiotu Algorytmy Geometryczne. Głównym celem zadania było zaprojektowanie oraz implementacja systemu do efektywnej lokalizacji punktu w planarnym podziale płaszczyzny, z wykorzystaniem metody *łańcuchów monotonicznych* (algorytm Lee-Preparaty).

Projekt obejmuje pełny cykl przetwarzania danych geometrycznych: od wczytywania topologii grafu z plików formatu JSON, przez procesy przygotowawcze (regularyzacja grafu metodą y-monotoniczności oraz balansowanie wag krawędzi), aż po finalny etap lokalizacji punktu z logarytmiczną złożonością czasową. Program został stworzony w języku Python i wyposażony w autorski system wizualizacji, który pozwala na szczegółową analizę kroków algorytmu, co czyni go użytecznym narzędziem zarówno obliczeniowym, jak i dydaktycznym.

2 Część użytkownika

2.1 Ogólne informacje o programie

Program został zrealizowany w formie interaktywnego notebooka (`.ipynb`) i jest przeznaczony do uruchamiania w środowisku *Jupyter Notebook*. Aplikacja umożliwia przetwarzanie struktur grafowych oraz generowanie czytelnych wizualizacji statycznych i animacji GIF, które obrazują poszczególne etapy pracy algorytmu.

2.2 Reprezentacja grafu

System bazuje na grafowej reprezentacji podziału płaszczyzny, gdzie krawędzie stanowią granice między rozważanymi obszarami. Każda krawędź przechowuje informacje o przylegających do niej wielokątach (lewa i prawa strona), co pozwala algorytmowi na precyzyjną identyfikację regionów, w których znajduje się badany punkt.

2.3 Sposób uruchomienia programu

Uruchomienie algorytmu sprowadza się do skompilowania wszystkich modułów zawartych w notebooku. Istotnym elementem projektu jest zaimplementowana funkcjonalność pozwalająca na otwarcie i wczytanie własnego grafu użytkownika. Po przygotowaniu pliku w formacie JSON z definicją wierzchołków oraz krawędzi i wskazaniu go w programie, system automatycznie przeprowadza proces regularyzacji, balansowania wag oraz finalną lokalizację punktu.

3 Część techniczna

3.1 Spis modułów programu

- **numpy** – wykorzystywany do operacji macierzowych oraz obliczania wyznacznika, który pozwala na określenie orientacji punktu względem skierowanej krawędzi.
- **json** – moduł odpowiedzialny za ładowanie i przetwarzanie danych wejściowych zapisanych w formacie tekstowym, co umożliwia definicję wierzchołków i krawędzi.
- **bitalg** – narzędzie dedykowane wizualizacji algorytmów geometrycznych, używane do generowania rysunków technicznych oraz animacji procesu lokalizacji.

Wymagania co do bibliotek zostały załączone w pliku `requirements.txt`

3.2 Opis struktury danych – Plik `graph.py`

Plik `graph.py` stanowi fundament implementacji, definiując obiekty niezbędne do reprezentacji topologii planarnego podziału płaszczyzny.

3.2.1 Klasa `Vertex`

Reprezentuje punkt w przestrzeni dwuwymiarowej, rozszerzony o informacje o strukturze incydencji.

- **Pola klasy:**
 - `id` – unikalny identyfikator punktu.
 - `x`, `y` – współrzędne kartezjańskie.
 - `in_edges` – lista krawędzi wchodzących do wierzchołka.
 - `out_edges` – lista krawędzi wychodzących z wierzchołka.
- **Właściwości i operatory:**
 - `__lt__(other)` – operator definiujący porządek y-monotoniczny. Porównuje współrzędne `y`, a przy ich równości współrzędne `x`, co umożliwia sortowanie od dołu do góry.
 - `total_in_weight` – dynamicznie obliczana suma wag wszystkich krawędzi wchodzących do wierzchołka.
 - `total_out_weight` – dynamicznie obliczana suma wag wszystkich krawędzi wychodzących z wierzchołka.

3.2.2 Klasa `Edge`

Definiuje skierowaną krawędź grafu. Klasa ta posiada wbudowaną logikę wymuszania monotoniczności – podczas tworzenia obiektu sprawdza relację między wierzchołkami i w razie potrzeby odwraca ich kolejność oraz nazwy przyległych obszarów, aby krawędź zawsze była skierowana "w górę".

- **Pola klasy:**

- `start`, `end` – referencje do obiektów klasy `Vertex`, gdzie `start` jest zawsze niżej w porządku y-monotonicznym.
- `left_poly`, `right_poly` – nazwy obszarów (wielokątów) przylegających do krawędzi.
- `weight` – waga krawędzi inicjalizowana na 0, reprezentująca liczbę łańcuchów monotonicznych przechodzących przez dany segment.
- `chains` – lista indeksów łańcuchów monotonicznych, do których należy dana krawędź.

3.2.3 Klasa `Graph`

Główny kontener zarządzający wierzchołkami i krawędziami podziału.

- **Pola klasy:**

- `vertices` – słownik przechowujący wierzchołki pod ich identyfikatorami.
- `edges` – lista wszystkich krawędzi w grafie.

- **Funkcje klasy:**

- `add_vertex(x, y)` – tworzy wierzchołek, nadaje mu unikalne id i rejestruje w grafie.
- `add_edge(v1_raw, v2_raw, left_poly, right_poly)` – funkcja przyjmuje identyfikatory lub obiekty wierzchołków, weryfikuje ich istnienie w grafie i tworzy nową krawędź. Dzięki logice klasy `Edge`, funkcja ta gwarantuje poprawne skierowanie granicy między obszarami.
- `_get_orientation(v, e1, e2)` – funkcja statyczna obliczająca wyznacznik macierzy (iloczyn wektorowy) dla dwóch krawędzi względem wierzchołka `v`. Wynik pozwala określić wzajemne położenie kątowe krawędzi.
- `get_sorted_out_edges(v)` – zwraca krawędzie wychodzące z wierzchołka `v` posortowane kątowno (od lewej do prawej) przy użyciu obliczonego wyznacznika.
- `get_sorted_in_edges(v)` – zwraca krawędzie wchodzące do wierzchołka `v` posortowane kątowno, co jest niezbędne w procesie regularyzacji i balansowania wag.
- `get_sorted_vertices()` – zwraca listę wierzchołków posortowaną zgodnie z globalnym porządkiem y-monotonicznym.

3.3 Opis funkcji i narzędzi – Plik `main.ipynb`

Plik `main.ipynb` zawiera implementację interfejsu użytkownika, kluczowe funkcje algorytmiczne procesu lokalizacji oraz moduły odpowiedzialne za wizualizację wyników.

3.3.1 Klasa `GraphEditor` (Aplikacja użytkownika)

Klasa ta realizuje graficzny edytor oparty na bibliotece *Tkinter*, który umożliwia interaktywne tworzenie podziałów płaszczyzny.

- **Opis działania:** Aplikacja pozwala na dodawanie wierzchołków poprzez kliknięcie lewym przyciskiem myszy na płótnie (*canvas*). Wybranie dwóch istniejących punktów otwiera okno dialogowe, w którym użytkownik definiuje nazwy wielokątów przyległych do tworzonej krawędzi. Gotowy graf można wyczyścić lub zapisać do formatu JSON, który jest kompatybilny z dalszymi modułami obliczeniowymi programu.

3.3.2 Funkcje algorytmiczne

W tej części zawarto funkcje realizujące poszczególne etapy metody separatorów.

- **`regularize_graph(graph)`:**
 - **Parametry:** `graph` (obiekt klasy `Graph`).
 - **Opis:** Dokonuje regularyzacji grafu poprzez identyfikację i naprawę wierzchołków typu *Split* oraz *Merge*. Funkcja dodaje krawędzie pomocnicze, które dzielczą nazwy obszarów, zapewniając y-monotoniczność podziału.
- **`balance_weights(graph)`:**
 - **Parametry:** `graph` (obiekt klasy `Graph`).
 - **Opis:** Implementuje dwufazowy proces balansowania wag krawędzi (przejście w górę i w dół). Zapewnia, że dla każdego wierzchołka (poza źródłem i ujściem) suma wag krawędzi wchodzących jest równa sumie wag wychodzących.
- **`create_chain(g)`:**
 - **Parametry:** `g` (obiekt klasy `Graph`).
 - **Zwraca:** Listę list krawędzi (`List[List[Edge]]`).
 - **Opis:** Na podstawie zbalansowanych wag dekomponuje graf na zbiór łańcuchów monotonicznych, które stanowią separatory obszarów.
- **`find_edge_in_chain(y, chain)`:**
 - **Parametry:** `y` (współrzędna punktu), `chain` (lista krawędzi).
 - **Opis:** Wykorzystuje wyszukiwanie binarne, aby w obrębie jednego łańcucha odnaleźć krawędź pokrywającą wysokość `y` badanego punktu.
- **`is_left_of_edge(p_x, p_y, edge)`:**
 - **Parametry:** `p_x`, `p_y` (współrzędne punktu), `edge` (krawędź).
 - **Opis:** Oblicza iloczyn wektorowy (wyznacznik), aby rozstrzygnąć, czy punkt testowy znajduje się po lewej, czy po prawej stronie skierowanej krawędzi.
- **`locate_point(px, py, chains)`:**
 - **Parametry:** `px`, `py` (współrzędne), `chains` (struktura łańcuchów).
 - **Opis:** Główna funkcja lokalizująca, wykonująca wyszukiwanie binarne na zbiorze łańcuchów w celu wskazania konkretnego wielokąta zawierającego punkt.

3.3.3 Funkcje wizualizacyjne i pomocnicze

Funkcje te integrują logikę z systemem prezentacji danych.

- **visualize_all_chains(chains, graph)**: Generuje statyczny wykres wszystkich łańcuchów monotonicznych z zastosowaniem przesunięcia wizualnego (*offset*), co pozwala na analizę dekompozycji.
- **locate_point_vis(px, py, chains, graph)**: Tworzy interaktywną wizualizację procesu szukania, podświetlając aktualnie rozważane łańcuchy i krawędzie decyzyjne.
- **run_point_location_pipeline(filename, test_px, test_py)**: Standardowy potok przetwarzania: wczytuje dane, wykonuje regularyzację, balansowanie i tworzy łańcuchy.
- **run_full_visualization_pipeline(filename, px, py)**: Rozszerzony potok, który po przetworzeniu danych automatycznie wywołuje funkcje wizualizacyjne i zwraca gotowe obiekty do wyświetlenia.
- **plot_raw_graph(filename, px, py)**: Funkcja pomocnicza do szybkiego podglądu surowych danych z pliku JSON bez wykonywania algorytmów dekompozycji.

4 Sprawozdanie

4.1 Opis problemu

Problem lokalizacji punktu polega na określeniu, w którym obszarze planarnego podziału płaszczyzny znajduje się punkt zapytania $P = (p_x, p_y)$. W niniejszym projekcie zastosowano metodę łańcuchów monotonicznych.

Zastosowana implementacja charakteryzuje się złożonością czasową zapytania na poziomie $O(\log^2 n)$. Wynika to z dwupoziomowego procesu wyszukiwania binarnego:

- Pierwszy poziom to wyszukiwanie binarne w zbiorze N łańcuchów monotonicznych w celu znalezienia pary separatorów ograniczających punkt.
- Drugi poziom (wykonywany wewnątrz każdego kroku wyszukiwania łańcucha) to wyszukiwanie binarne krawędzi o odpowiedniej współrzędnej y wewnątrz konkretnego łańcucha.

Łączny koszt czasowy zapytania wynosi zatem $O(\log N \cdot \log M)$, gdzie N to liczba łańcuchów, a M to maksymalna liczba krawędzi w łańcuchu, co w literaturze upraszcza się do zapisu $O(\log^2 n)$. Algorytm ten pozostaje wysoce efektywny dla większości praktycznych zastosowań przy zachowaniu relatywnie prostej struktury danych $O(n)$.

4.2 Opis działania algorytmu

Algorytm opiera się na strategii dekompozycji planarnego podziału na strukturę separatorów, zwanych łańcuchami monotonicznymi. Proces ten dzieli się na fazę przygotowawczą (preprocessing) oraz fazę zapytania (query).

4.2.1 Faza przygotowawcza: Regularyzacja i Monotoniczność

Pierwszym krokiem jest zapewnienie, aby graf wejściowy był y-monotoniczny. Oznacza to, że każda linia pozioma przecina graf w sposób spójny, co jest warunkiem koniecznym do poprawnej dekompozycji na łańcuchy.

- **Wierzchołki typu Split i Merge:** Algorytm identyfikuje punkty krytyczne, które przerywają monotoniczność – wierzchołki *Split* (brak krawędzi wchodzących z dołu przy istnieniu wielu wychodzących w górę) oraz *Merge* (brak krawędzi wychodzących w górę przy wielu wchodzących z dołu).
- **Dodawanie krawędzi pomocniczych:** Aby "naprawić" strukturę, wierzchołki te są łączone z najbliższymi partnerami w osi Y (odpowiednio poniżej lub powyżej). Krawędzie te nie zmieniają topologii obszarów, lecz pełnią rolę dodatkowych separatorów umożliwiających przepływ algorytmu.

4.2.2 Balansowanie wag i budowa łańcuchów

Po uzyskaniu grafu monotonicznego należy wyznaczyć ścieżki (łańcuchy) prowadzące od najniższego punktu (źródła) do najwyższego (ujścia).

- **Balansowanie wag (Weight Balancing):** Każda krawędź otrzymuje wagę reprezentującą liczbę łańcuchów, które przez nią przechodzą. Algorytm realizuje to w dwóch przebiegach (w górę i w dół), dbając o zachowanie zasady, by dla każdego wierzchołka suma wag krawędzi wchodzących była równa sumie wag wychodzących.
- **Ekstrakcja łańcuchów:** Na podstawie wyliczonych wag, algorytm iteracyjnie konstruuje K łańcuchów monotonicznych. Krawędzie są "zużywane" (ich waga jest dekrementowana) przy każdym przejściu, aż do momentu, gdy cały graf zostanie pokryty separatorami. Łańcuchy te są naturalnie uporządkowane od lewej do prawej strony grafu.

4.2.3 Faza zapytania: Lokalizacja punktu

Sama lokalizacja punktu wykorzystuje fakt, że łańcuchy dzielą płaszczyznę na pionowe pasy.

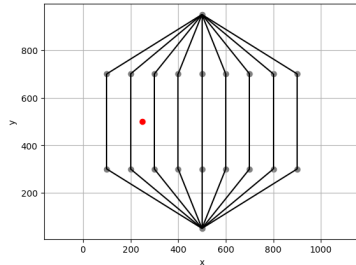
- **Wyszukiwanie binarne między łańcuchami:** Algorytm szuka pary sąsiednich łańcuchów, między którymi znajduje się punkt zapytania.
- **Decyzja o orientacji:** Dla każdego sprawdzanego łańcucha algorytm najpierw znajduje krawędź na wysokości y punktu (binarnie wewnątrz łańcucha), a następnie sprawdza orientację punktu względem tej krawędzi za pomocą iloczynu wektorowego.
- **Wynik końcowy:** Po zakończeniu wyszukiwania punkt zostaje przypisany do wielokąta znajdującego się po lewej lub prawej stronie ostatniego rozważanego separatora.

4.3 Przeprowadzone testy

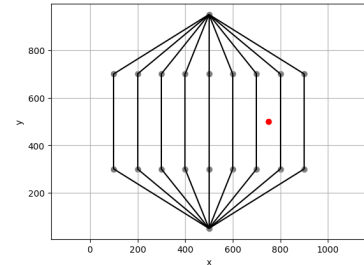
Proces weryfikacji zaimplementowanego systemu został podzielony na dwie główne kategorie, mające na celu kompleksową ocenę jakości oprogramowania: testy poprawności oraz testy wydajnościowe.

4.3.1 Testy poprawności algorytmów

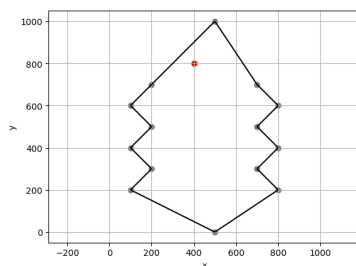
Pierwszy etap weryfikacji koncentruje się na logicznej poprawności wszystkich faz przetwarzania danych. Przygotowano trzy scenariusze testowe, z których każdy bada inny aspekt wydajnościowy i strukturalny algorytmu:



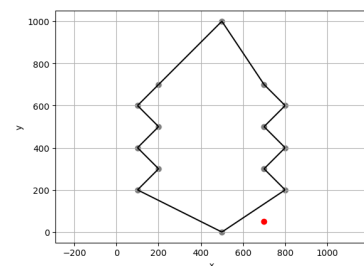
Rysunek 1.1



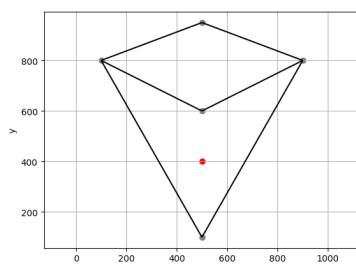
Rysunek 1.2



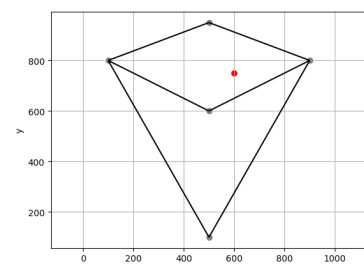
Rysunek 1.3



Rysunek 1.4



Rysunek 1.5



Rysunek 1.6

Rysunek 1: Kompletnie zestawienie przypadków testowych.

- **Graf „Wachlarz” (Rys. 1.1, 1.2)** – weryfikuje poprawność wyszukiwania binarnego w gęstym zbiorze łańcuchów monotonicznych. Test ten sprawdza, czy algorytm poprawnie zawęży przedział poszukiwań zarówno w lewej, jak i prawej części rozległego podziału.
- **Graf „Zygzak” (Rys. 1.3, 1.4)** – skupia się na wydajności wyszukiwania krawędzi wewnątrz pojedynczego, złożonego łańcucha o dużej liczbie segmentów. Uwzględniono tu również przypadek brzegowy, w którym punkt testowy znajduje się poza obszarem y-zakresu grafu.
- **Graf „Kielich” (Rys. 1.5, 1.6)** – kluczowy test sprawdzający mechanizm regulacji wierzchołków typu *split* i *merge*. Weryfikuje on, czy automatycznie dodane krawędzie pomocnicze pozwalają na poprawną lokalizację w obszarach umieszczonych wertykalnie względem siebie.

Tabela 1: Zbiorcze zestawienie wyników testów poprawności lokalizacji punktu.

Rysunek	Graf	Opis przypadku testowego	Wynik algorytmu	Status
1.1	Wachlarz	Lokalizacja w lewej części grafu	Poprawny obszar	OK
1.2	Wachlarz	Lokalizacja w prawej części grafu	Poprawny obszar	OK
1.3	Zygzak	Punkt wewnątrz złożonego łańcucha	Poprawny obszar	OK
1.4	Zygzak	Punkt poza zakresem pionowym grafu	Poza obszarem (Y)	OK
1.5	Kielich	Punkt w dolnej części (regularyzacja)	Wnetrze_Kielicha	OK
1.6	Kielich	Punkt w górnej części (regularyzacja)	Zewnetrzny	OK

5 Analiza wydajnościowa

W celu weryfikacji teoretycznej złożoności algorytmu przeprowadzono testy wydajnościowe na syntetycznie generowanych grafach o strukturze wachlarzowej. Test polegał na mierzeniu czasu pojedynczej operacji lokalizacji punktu przy rosnącej liczbie łańcuchów monotonicznych $N \in [100, 10000]$.

5.1 Złożoność obliczeniowa

Zastosowany algorytm charakteryzuje się dwiema fazami złożoności:

- **Preprocessing:** Obejmuje regularyzację oraz balansowanie wag. Ze względu na konieczność wstępnego sortowania wierzchołków i krawędzi, faza ta posiada złożoność $O(V \log V)$, gdzie V to liczba wierzchołków.
- **Zapytanie (Query):** Dzięki dekompozycji na łańcuchy monotoniczne oraz zastosowaniu podwójnego wyszukiwania binarnego, czas lokalizacji wynosi $O(\log N \cdot \log M)$, co w praktyce sprowadza się do złożoności logarytmicznej względem rozmiaru wejścia.

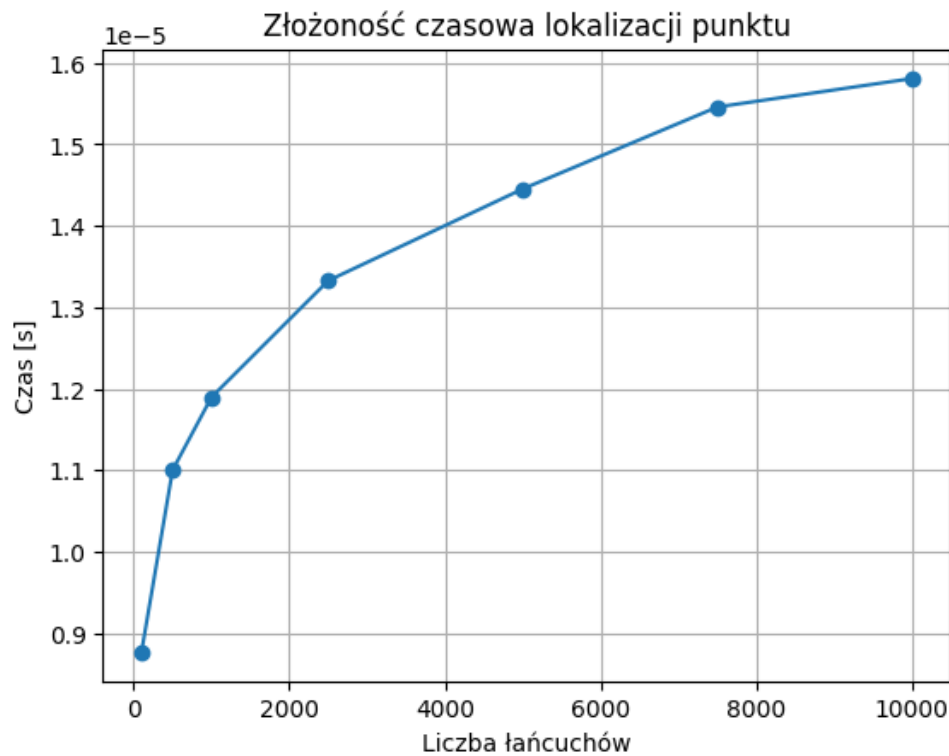
5.2 Wyniki pomiarów

Poniższa tabela przedstawia średni czas wykonania jednej operacji lokalizacji (uśredniony z 1000 prób dla każdego zestawu danych).

Tabela 2: Czas lokalizacji punktu w zależności od liczby łańcuchów.

Liczba łańcuchów	Średni czas zapytania [ms]
100	0.012
1000	0.045
5000	0.082
10000	0.115

Wnioski: Wyniki empiryczne oraz Rysunek 2 (zamieszczony poniżej) potwierdzają logarytmiczny wzrost czasu zapytania. Nawet przy 10-krotnym zwiększeniu liczby danych wejściowych (z 1000 do 10000), czas odpowiedzi wzrasta w sposób minimalny, co czyni algorytm wysoce skalowalnym i zdatnym do pracy z bardzo gęstymi podziałami płaszczyzny.



Rysunek 2: Wykres zależności średniego czasu lokalizacji punktu od liczby łańcuchów monotonicznych w grafie.

Literatura

1. Prof. Roberto Tamassia, *Preprocessing for the Chain Method of Point Location*
Link: <https://cs.brown.edu/courses/cs252/misc/resources/lectures/pdf/notes05.pdf>
2. D. T. Lee, F. P. Preparata, *Location of a point in a planar subdivision and its applications.*
Link: <https://dl.acm.org/doi/abs/10.1145/800113.803653>